

STRUCTURES ET FONCTIONS

RAPPEL : DÉFINITION D'UNE STRUCTURE

```
struct [nom_struct] {  
    Type1 membre1 ;  
    ...  
    TypeN membreN ;  
} [var1, ..., varm] ;
```

Une **structure** rassemble des variables (nommés membres de la structure) qui peuvent être de type différents (hétérogènes) sous un seul nom.

Cela permet une manipulation plus facile de variables liées.

L'accès aux membres de la structure se fera via le schéma suivant :

Identificateur • nomMembre

L'opérateur • sert d'opérateur de sélection aux différents membres de la structure.

PASSAGE D'UNE STRUCTURE EN PARAMÈTRE À UNE FONCTION

Deux solutions sont possibles pour passer une structure en paramètre à une fonction :

- A) Passage par valeur

- B) Passage par adresse

PASSAGE PAR VALEUR

Une structure peut être **passée en paramètre** à une fonction. Comme tout paramètre, elle est recopiée localement à l'entrée de la fonction.

En effet, une structure une fois déclarée s'utilise quasiment comme une autre variable ; par exemple, le passage dans une fonction **se fera aussi par valeur** (ce sont les valeurs des membres qui seront passées).

Exemple

```
typedef struct complexe {  
    double preel ;  
    double pima ;  
} Complexe;
```

```
typedef struct {  
    double preel ;  
    double pima ;  
} Complexe;
```

Rappel : lorsqu'on affecte la « valeur » d'une structure à une autre, cela revient à affecter chacun des membres de la première aux membres de la seconde.

$$c1 = c2 \Leftrightarrow \begin{cases} c1.preel = c2.preel \\ c1.pima = c2.pima \end{cases}$$

// Affichage d'un complexe

```
void afficheComplexe(Complexe c) {
```

```
    printf(" \n C=%lf +i*%lf ", p.reel, p.ima);
```

```
}
```

Fournit à la procédure `afficheComplexe` une copie de la structure `Complexe`.

Appel de la procédure :

`afficheComplexe(c)`

PASSAGE D'UNE STRUCTURE PAR ADRESSE

Une fonction peut prendre aussi l'adresse d'une structure en paramètre.

C'est cette deuxième solution qu'elle est toujours conseillé d'utilisée.

Ceci évite la duplication de la structure sur la pile (opération qui peut être coûteuse, voire impossible si la structure occupe une taille mémoire importante).

Accès aux membres d'une structure pointée

Complexe *p; // p est un pointeur sur une structure

Pour accéder aux différents membres de la structure pointée Complexe par exemple on utilise les notations suivantes :

p-> Ou (*p)•

- * donne le contenu du pointeur
- et l'opérateur • permet l'accès aux différents membres de la structure.

Exemple

// Affichage d'un complexe

```
void afficheComplexe(Complexe *c) {  
  
    printf(" \n C=%lf +i*%lf " , p→reel, p→ima);  
  
    //où printf(" \n C=%lf +i*%lf " , (*p).reel, (*p). ima);  
  
}
```

Fournit à la procédure `afficheComplexe` l'adresse de la structure `Complexe`.

Appel de la procédure :

`afficheComplexe(&c)`

FONCTION RETOURNANT UNE STRUCTURE

```
Complexe initComplexe(double r, double i){
```

```
Complexe c ;
```

```
    c.reel = r ;
```

```
    c.ima = i ;
```

```
    return c ;
```

```
}
```

UTILISATION DES PASSAGES PAR VALEUR

// Somme de deux complexes

// somme de deux complexes en tant que fonction

```
Complexe fsommeComplexe(Complexe c1, Complexe c2){
```

```
Complexe s ;
```

```
    s.reel = c1.reel + c2.reel ;
```

```
    s.ima = c1.ima + c2.ima ;
```

```
    return s ;
```

```
}
```

// somme de deux complexes en tant que procédure

```
void psommeComplexe(Complexe c1, Complexe c2, Complexe *s){
```

```
    (*s).reel = c1.reel + c2.reel ;
```

```
    (*s).ima = c1.ima + c2.ima ;
```

```
}
```


Exemple d'Appels

```
{  
...  
    // Initialisation  
    c1 = initComplexe(10, 20) ;  
  
    c2 = initComplexe(20, 10) ;  
  
    // Affichage de la somme de deux complexes  
  
    //Utilisation de la somme en tant que fonction  
    afficheComplexe( fsommeComplexe(c1, c2) ) ;  
  
    //Utilisation de la somme en tant que procédure  
    psommeComplexe(c1, c2, &c) ;  
    afficheComplexe( c) ;  
  
    ...  
}
```

UTILISATION DES PASSAGES PAR ADRESSE

// Somme de deux complexes

// Somme de deux complexes en tant que fonction

```
Complexe fsommeComplexe(Complexe *c1, Complexe *c2){  
    Complexe s ;  
    s.reel = c1→reel + c2→reel ; // où s.reel = (*c1).reel + (*c2).reel ;  
    s.ima = c1→ima + c2→ima ; // où s.ima = (*c1).ima + (*c2).ima ;  
    return s ;  
}
```

// Somme de deux complexes en tant que procédure

```
void psommeComplexe(Complexe *c1, Complexe *c2, Complexe *s){  
    s→reel = c1→reel + c2→reel ; //où (*s).reel = (*c1).reel + (*c2).reel ;  
    s→ima = c1→ima + c2→ima ; //où (*s).ima = (*c1).ima + (*c2).ima ;  
}
```

Exemple d'Appels

```
{  
...  
    // Initialisation  
    c1 = initComplexe(10, 20) ;  
    c2 = initComplexe(20, 10) ;  
    // Affichage de la somme de deux complexes  
  
    //Utilisation de la somme en tant que fonction  
    s= fsommeComplexe(&c1, &c2) ;  
  
    afficheComplexe( &s ) ;  
  
    //Utilisation de la somme en tant que procédure  
    psommeComplexe(&c1, &c2, &s) ;  
  
    afficheComplexe( &s) ;  
  
    ...  
}
```